LAB NO. 03

CONSTRUCTOS AND DESTRUCTORS IN C++

Lab Objectives

Following are the lab objectives:

1. Learn to define constructors and destructors in classes
2. Learn to define constrctors and destructors types in classes

Student ID	9864	Student Name	Ali Raza
	Marks	**Obtained Marks**	**Comments**
Task 1	**15**		
Task 2	**15**		
Task 3	**15**		
Total Marks	**45**		

Lab Instructor

Lab Objectives and CLOs Mapping

Lab Objectives	CLOs		
	a	b	c
1			
2			
3			

Instructions

- This is individual Lab work/task.
- Complete this lab work within lab timing.
- Discussion with peers is not allowed.
- You can consult any book, notes & Internet.
- Copy paste from Internet will give you **negative marks**.
- Lab work is divided into small tasks, complete all tasks sequentially.
- Show solution of each lab task to your Lab Instructor.
- In-Lab Exercises/Tasks
- **Write your code at provided space after each question**
- **You need to upload code for all tasks at Google Class.**

Constructor and Destructor in C++

Constructor and Destructor are the special member functions of the class which are created by the C++ compiler or can be defined by the user. Constructor is used to initialize the object of the class while destructor is called by the compiler when the object is destroyed.

What is Constructor in C++?

A constructor is a special member function of a class and shares the same name as of class, which means the constructor and class have the same name. Constructor is called by the compiler whenever the object of the class is created, it allocates the memory to the object and initializes class data members by default values or values passed by the user while creating an object. Constructors don’t have any return type because their work is to just create and initialize an object.

What is the basic syntax of Constructor in C++?

The basic syntax of the constructor is given below:

```
class class_name{

  private:

  // private members




public:



  // declaring constructor

  class_name({parameters})
```

```
  {

    // constructor body

  }



};
```

In the above syntax, we can see the class has the name class_name and the constructor have also the same name. A constructor can have any number of parameters as per requirements. Also, there is no return type or return value of the constructor.
Note: In the above syntax we have declared the constructor as a public member but we can declare it private also (we will discuss that later in this article).

Important Points about Constructors

Access specifiers

Constructors can be defined as public, protected, or private as per the requirements. By default or default constructors, which are created by the compiler are declared as the public. If the constructor is created as private, then we are not able to create its object.
When there is no requirement of the object of a class (in a situation when all class members are static) we can define its constructor as private.
Usually, constructors are defined as public because constructors are used to create an object and initialize the class data members for the object. An object is always created from outside of class, which justifies making constructors public.

Inheritance

As a derived class can access all the public properties of the base class, it can call its constructor also if it is not declared as private. Also, the constructor's address cannot be referenced.

Virtual

Constructor in C++ cannot be declared as virtual because when we declare any function of a class as a virtual function, at compile time compiler creates a virtual table to store the address of each function that is declared as virtual. Also, it creates a data member of the class virtual pointer to points towards the virtual table. But as we have discussed we cannot refer to the address of the constructor, which means we are not able to declare the constructor as virtual.

How many types of Constructors are present in C++?

There are four types of constructors in c++

- Default constructor
- Parameterized constructor
- Copy Constructor
- Dynamic Constructor

Let's discuss them in detail

1. Default Constructor

Default constructor is also known as a zero-argument constructor, as it doesn't take any parameter. It can be defined by the user if not then the compiler creates it on his own. Default constructor always initializes data members of the class with the same value they were defined.

Syntax

```
class class_name{

  private:

  // private members



  public:
```

```
  // declaring default constructor
  class_name()
  {
    // constructor body
  }

};
```

Code to show the working of default constructor

```
#include <iostream>
using namespace std;

class Person{

  // declaring private class data members
private:
  string name;
  int age;
```

```
public:


  // declaring constructor
  Person()
  {
      cout<<"Default constructor is called"<<endl;
      name = "student";
      age = 12;
  }


  // display function to print the class data members value
  void display()
  {
      cout<<"Name of current object: "<<name<<endl;
      cout<<"Age of current object: "<<age<<endl;
  }
```

```
};

int main()

{

    // creating object of class using default constructor

    Person obj;



    // printing class data members

    obj.display();



    return 0;

}
```

Output

```
Default constructor is called

Name of current object: student

Age of current object: 12
```

In the above code, we have created a class with two data members. Declared a default constructor which always initializes objects of a class with the same name and age. In the main function, we have created an object of the class and printed its data member values by using the display function.

2. Parameterized Constructor

Parameterized constructor is used to initialize data members with the values provided by the user. This constructor is basically the upgraded version of the default constructor. We can define more than one parameterized constructor according to the need of the user, but we have to follow the rules of the function overloading, like a different set of arguments must be there for each constructor.

Syntax

```
class class_name{

  private:

  // private members



  public:



  // declaring parameterized constructor

  class_name(parameter1, parameter2,...)

  {

    // constructor body

  }



};
```

Code to understand the working of the parameterized constructor

```
#include <iostream>

using namespace std;



class Person{



  // declaring private class data members

private:

  string name;

  int age;



public:



  // declaring parameterized constructor of three different types

  Person(string person_name)

  {

      cout<<"Constructor to set name is called"<<endl;

      name = person_name;
```

```
    age = 12;
}

Person(int person_age)
{
    cout<<"Constructor to set age is called"<<endl;
    name = "Student";
    age = person_age;
}

Person(string person_name, int person_age)
{
    cout<<"Constructor for both name and age is called"<<endl;
    name = person_name;
    age = person_age;
}
// display function to print the class data members value
void display()
```

```
  {
      cout<<"Name of current object: "<<name<<endl;
      cout<<"Age of current object: "<<age<<endl;
      cout<<endl;
  }

};
int main()
{
    // creating objects of class using parameterized constructor
    Person obj1("First person");

    // printing class data members for first object
    obj1.display();

    Person obj2(25);

    // printing class data members for second object
```

```
    obj2.display();


    Person obj3("Second person",15);


    // printing class data members for third object
    obj3.display();
    return 0;
}
```

Output

```
Constructor to set name is called
Name of current object: First person
Age of current object: 12


Constructor to set age is called
Name of current object: Student
Age of current object: 25


Constructor for both name and age is called
```

```
Name of current object: Second person


Age of current object: 15
```

In the above code, we have created three types of the parametric constructor, one for initialization of name only, second to initialization of age only, and third to initialize both name and age. In the main function, we have created three different types of objects and initialized them in different ways, and printed values for each of them.

3. Copy Constructor

If we have an object of a class and we want to create its copy in a new declared object of the same class, then a copy constructor is used. The compiler provides each class a default copy constructor and users can define it also. It takes a single argument which is an object of the same class.

Syntax

```
class class_name{

  private:

  // private members



  public:



  // declaring copy constructor

  class_name(const class_name& obj)
```

```
  {

    // constructor body

  }



};
```

In the above syntax, we created a copy constructor which takes an object of the same class as a parameter but it is declared constant and passed as a reference because when an argument is passed as a function parameter it creates a copy for it, to create that copy compiler will again call the copy constructor, means it will call the same function and for that call again there will be a call to create copy which will take this process in neverending recursion of creating copies. To prevent such conditions we pass it as a reference.

Code to understand the working of the copy constructor

```
#include <iostream>

using namespace std;



class Person{



  // declaring private class data members

private:

  string name;
```

```
  int age;



public:

  Person(string person_name, int person_age)

  {

      cout<<"Constructor for both name and age is called"<<endl;

      name = person_name;

      age = person_age;

  }



  Person(const Person& obj)

  {

      cout<<"Copy constructor is called"<<endl;

      name = obj.name;

      age = obj.age;

  }

  // display function to print the class data members value

  void display()
```

```
  {
      cout<<"Name of current object: "<<name<<endl;
      cout<<"Age of current object: "<<age<<endl;
      cout<<endl;
  }

};
int main()
{
    // creating objects of class using parameterized constructor
    Person obj1("First person",25);

    // printing class data members for first object
    obj1.display();

    // creating copy of the obj1
    Person obj2(obj1);
```

```
    // printing class data members for second object

    obj2.display();



    return 0;

}
```

Output

```
Constructor for both name and age is called

Name of current object: First person

Age of current object: 25



Copy constructor is called

Name of current object: First person

Age of current object: 25
```

In the above code, we have created a class and defined two types of constructors in it, the first is a parameterized constructor and another is a copy constructor. Parameterized constructor is used to create an object then by using the copy constructor we create a copy of it and stored it in another object.

4. Dynamic Constructor

When memory is allocated dynamically to the data members at the runtime using a new operator, the constructor is known as the dynamic constructor.

This constructor is similar to the default or parameterized constructor; the only difference is it uses a new operator to allocate the memory.

Syntax

```
class class_name{

  private:

  // private members



  public:



  // declaring dynamic constructor

  class_name({parameters})

  {

    // constructor body where data members are initialized using new operator

  }



};
```

Code to understand the working of the dynamic constructor

```
#include <iostream>
```

```
using namespace std;



class Person{



  // declaring private class data members

private:

  int* age;



public:

  Person(int* person_age)

  {

      cout<<"Constructor for age is called"<<endl;



      // allocating memory

      age = new int;

      age = person_age;

  }
```

```
  // display function to print the class data members value
  void display()
  {
      cout<<"Age of current object: "<<*age<<endl;
      cout<<endl;
  }

};
int main()
{
    // creating objects of class using parameterized constructor
    int age = 25;
    Person obj1(&age);

    // printing class data members for first object
    obj1.display();

    return 0;
```

```
}
```

Output

```
Constructor for age is called

Age of current object: 25
```

In the above code, we have created a class with a dynamic constructor. In the main function, we have created an object and initialized it using a dynamic constructor, where we have given memory dynamically using a new operator.

What is Destructor in C++?

Destructor is just the opposite function of the constructor. A destructor is called by the compiler when the object is destroyed and its main function is to deallocate the memory of the object. The object may be destroyed when the program ends, or local objects of the function get out of scope when the function ends or in any other case.
Destructor has the same as of the class with prefix tilde(~) operator and it cannot be overloaded as the constructor. Destructors take no argument and have no return type and return value.

What is the Basic Syntax of the Destructor?

The basic syntax of the Destructor is given below

```
class class_name{

  private:

  // private members



  public:
```

```
  // declaring destructor

  ~class_name()

  {

    // destructor body

  }



};
```

In the above syntax, we can see the class has the name class_name and the destructor also has the same name, in addition there is a tilde(~). Also, there is no return type and return value of the destructor.
Note: In the above syntax we have declared the destructor as a public member but we can declare it private also.

How Constructor and Destructor are called when the object is Created and Destroyed

As constructor is the first function called by the compiler when an object is created and the destructor is the last class member called by the compiler for an object. If the constructor and destructor are not declared by the user, the compiler defines the default constructor and destructor of a class object.
Let's see a code to get the proper idea of how constructor and destructor are called:
First, we will create a class with single parametrized constructors and a destructor. Both of them contain print statements to give an idea of when they are called.

```
#include <iostream>

using namespace std;
```

```
class class_name{


  // declaring private class data members
private:
  int a,b;


public:


  // declaring Constructor
  class_name(int aa, int bb)
  {
      cout<<"Constructor is called"<<endl;
      a = aa;
      b = bb;


      cout<<"Value of a: "<<a<<endl;
      cout<<"Value of b: "<<b<<endl;
      cout<<endl;
```

```
  }


  // declaring destructor
  ~class_name()
  {
      cout<<"Destructor is called"<<endl;
      cout<<"Value of a: "<<a<<endl;
      cout<<"Value of b: "<<b<<endl;
  }


};
int main()
{
    // creating objects of class using parameterized constructor
    class_name obj(5,6);


    return 0;
}
```

LAB TASKS

Task 1 (15 mark)

Scenario: A **Library** needs to store details of books. Some books are newly added without full info, while others are entered with complete details.

- Create a class **Book** with members:
  - `string title`
  - `string author`
  - `int year`

Define:

- A **default constructor** that sets title = `"Unknown"`, author = `"Not Assigned"`, year = 0.
- A **parameterized constructor** that initializes all members.

- A `display()` function to print book info.

Main:

- Create one object with the default constructor (new book, no info yet).
- Create another with the parameterized constructor (complete book info).
- Display both.

(Past your code here.)

PROGRAM:

```
#include <iostream>

#include <string>

using namespace std;

class Book {

private:

    string title;

    string author;

    int year;



public:



    Book() {

        title = "Raza";

        author = "By Raza";

        year = 2000;
```

```
    }
    Book(string t, string a, int y) {
        title = t;
        author = a;
        year = y;
    }
    void display() {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Year: " << year << endl;
        cout << "" << endl;
    }
};
int main() {
    Book newBook;


    Book completeBook("2002", "Ali Raza", 2006);
    cout << "New Book Entry:" << endl;
    newBook.display();
```

```
    cout << "Complete Book Entry:" << endl;

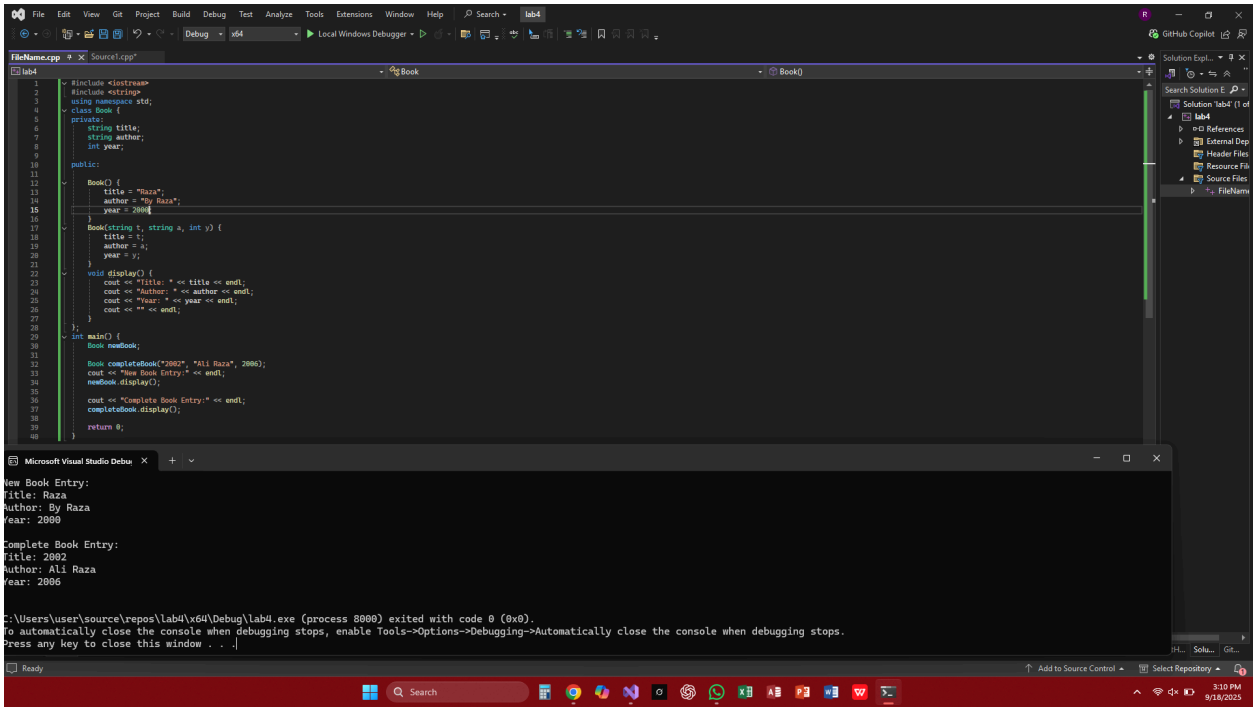
    completeBook.display();



    return 0;

}
```

OUTPUT:



Task 2

Scenario: An **Online Shopping System** sometimes needs to copy details of an existing product into another object (e.g., when creating a duplicate listing).

- Create a class **Product** with members:
    - `string name`
    - `double price`
    - `string category`

Define:

- A **parameterized constructor** to set product details.
- A **copy constructor** that copies details from another object.
- A `display()` function.

Main:

- Create a product using the parameterized constructor.
- Create another product using the **copy constructor**.
- Display both and show that the copy worked.

(Past your code here.)

Program:

```
#include <iostream>

using namespace std;



class Product {

private:
```

```
    string name;
    double price;
    string category;
public:
    Product(string n, double p, string c) {
        name = n;
        price = p;
        category = c;
    }

    Product(const Product& other) {
        name = other.name;
        price = other.price;
        category = other.category;
    }

    void display() const {
     cout << "Product Name: " << name << endl;
      cout << "Price: " << price << endl;
      cout << "Category: " << category << endl;
```

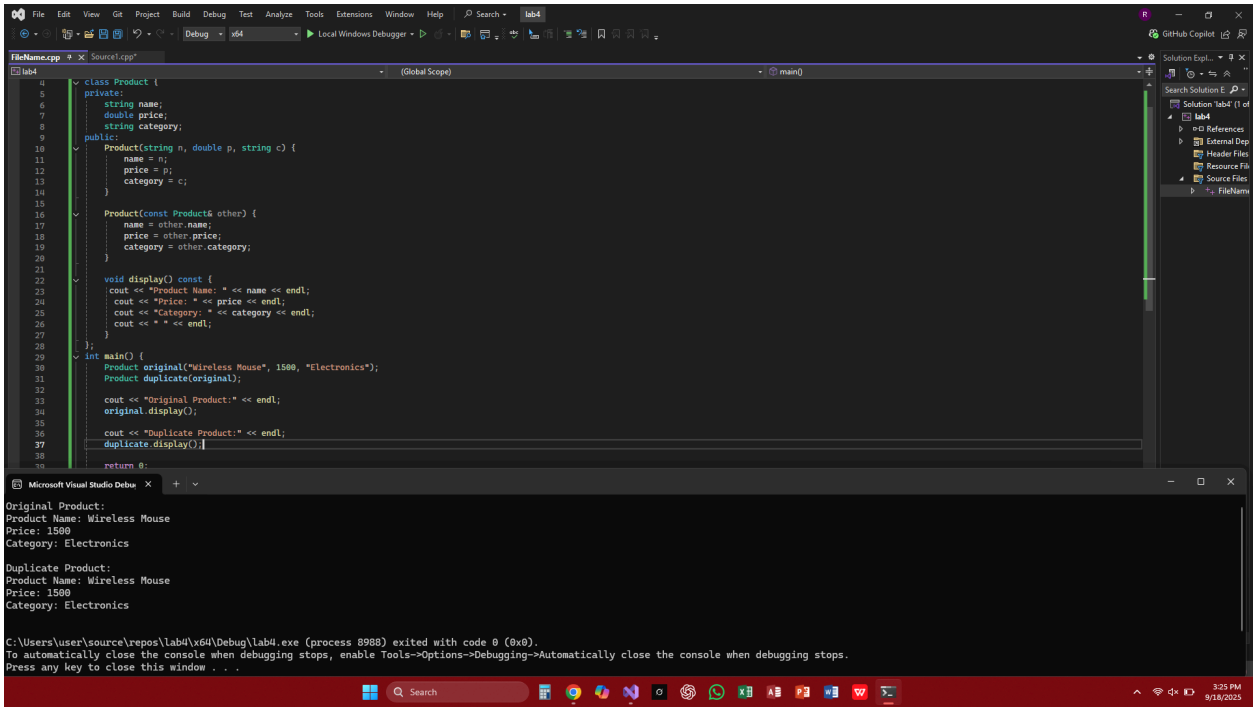
```
    cout << " " << endl;
  }
};
int main() {
  Product original("Wireless Mouse", 1500, "Electronics");
  Product duplicate(original);

  cout << "Original Product:" << endl;
  original.display();

  cout << "Duplicate Product:" << endl;
  duplicate.display();

  return 0;
}
```

Output:


class Product {
private:
string name;
double price;
string category;
public:
Product(string n, double p, string c) {
name = n;
price = p;
category = c;
}
Product(const Product& other) {
name = other.name;
price = other.price;
category = other.category;
}
void display() const {
cout << "Product Name: " << name << endl;
cout << "Price: " << price << endl;
cout << "Category: " << category << endl;
cout << " " << endl;
}
};
int main() {
Product original("Wireless Mouse", 1500, "Electronics");
Product duplicate(original);
cout << "Original Product:" << endl;
original.display();
cout << "Duplicate Product:" << endl;
duplicate.display();
return 0;
Original Product:
Product Name: Wireless Mouse
Price: 1500
Category: Electronics
Duplicate Product:
Product Name: Wireless Mouse
Price: 1500
Category: Electronics
C:\Users\user\source\repos\Lab4\x64\Debug\lab4.exe (process 8988) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

Task 3

Scenario: A **Bank System** automatically closes accounts when the program ends.

- Create a class **BankAccount** with members:
  - `string accountHolder`
  - `double balance`

Define:

- A **parameterized constructor** that initializes account details.
- A **destructor** that prints:
  `"Account of <accountHolder> with balance <balance> is now closed."`

Main:

- Create two bank account objects.
- When the program ends, observe how destructors are called automatically.

(Past your code here.)

Program:

```
#include <iostream>

#include <string>

using namespace std;



class BankAccount {

private:

   string accountHolder;

   double balance;



public:

   BankAccount(string holder, double bal) {
```

```
        accountHolder = holder;
        balance = bal;
        cout << "Account created for " << accountHolder << " with balance " << balance << endl;
    }

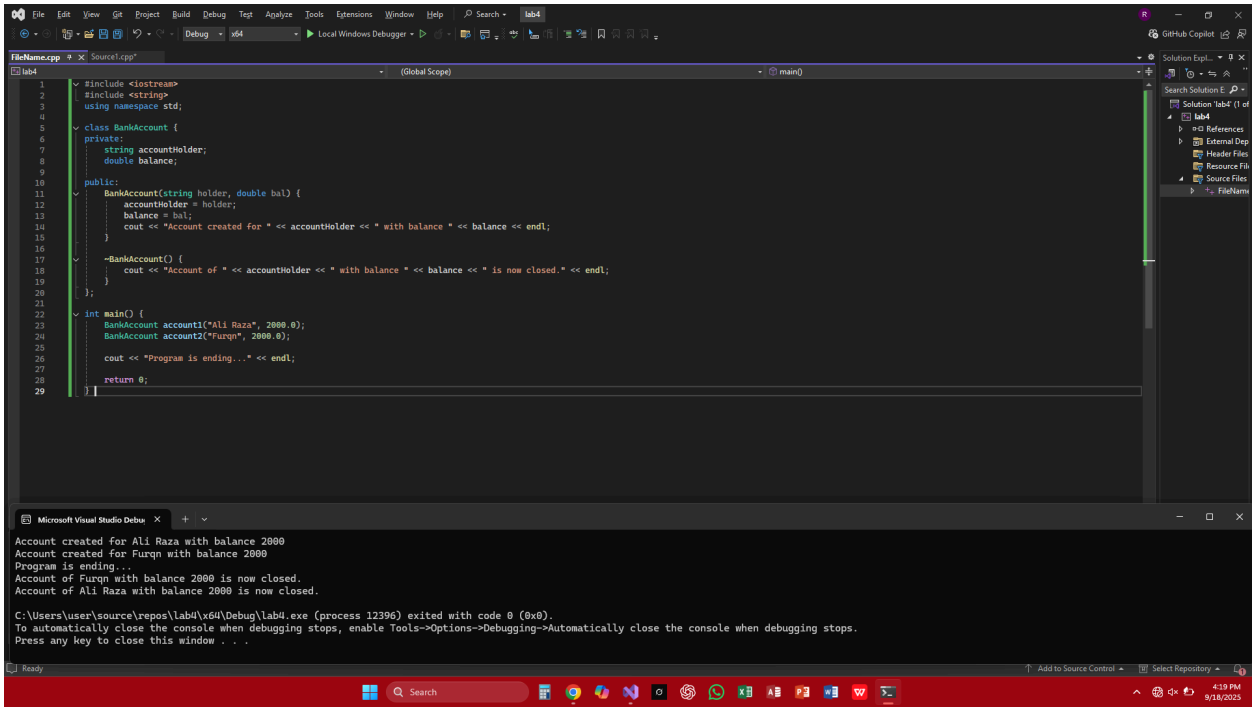
    ~BankAccount() {
        cout << "Account of " << accountHolder << " with balance " << balance << " is now closed." << endl;
    }
};

int main() {
    BankAccount account1("Ali Raza", 2000.0);
    BankAccount account2("Furqn", 2000.0);

    cout << "Program is ending..." << endl;

    return 0;
}
```

Output:


#include <iostream>
#include <string>
using namespace std;
class BankAccount {
private:
string accountHolder;
double balance;
public:
BankAccount(string holder, double bal) {
accountHolder = holder;
balance = bal;
cout << "Account created for " << accountHolder << " with balance " << balance << endl;
}
~BankAccount() {
cout << "Account of " << accountHolder << " with balance " << balance << " is now closed." << endl;
}
};
int main() {
BankAccount account1("Ali Raza", 2000.0);
BankAccount account2("Furqn", 2000.0);
cout << "Program is ending..." << endl;
return 0;
}
Account created for Ali Raza with balance 2000
Account created for Furqn with balance 2000
Program is ending...
Account of Furqn with balance 2000 is now closed.
Account of Ali Raza with balance 2000 is now closed.
C:\Users\user\source\repos\lab4\x64\Debug\lab4.exe (process 12396) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .